LARGE SCALE AND MULTISTRUCTURE DATABASE REPORT

GAMEHUB APPLICATION

Sommario

Introduction and Requirements 2
- Application Functionalities 2
- Functional Requirements 2
- Non Functional Requirements 2

Schemas 3
- Use-Case Diagram 3
- UML Diagram 4

Software Architecture 4
- Client Side 4
- Server Side 8

Dataset 10

MongoDB Design and Implementation 11
- Documents Structure 11
- Mongo Queries Analysis 13
  - Update Game's Review 14
- Aggregation Implementation 15
- Index Configurations 17

Sharding Considerations 18

Neo4j Design and Implementation 18
- Nodes 18
- Relathionships 18
- Neo4j Queries Analysis 21
- Neo4 Queries Implementation 22

Structure of the Packages 23

References 24

Introduction and Requirements

Application Functionalities

The application named *Gamehub* is developed as a web-application whose main functionalities consists in collecting games, presenting informations about the games to the users, write reviews about the games and follow other users. The informations about the game consists in name,genres,score, wich are the filters that the users can use to retrieve games from the database, and other detailed informations about the results they want to achieve and they can add those games to their gameList. The users can write reviews related to videogames with a comment and a rate, they can like other users's reviews and furthermore they can follow other users to visualize the wishlist of their own friends. In the application are performed some statistics wich can be called by the users and analytics wich can be called by the administrator who only have the permission for specific advanced queries

Functional Requirements

The users of the application are divided in: *Standard Users* who can access to the application with a login based on username and password or a signup system if they are new users, and they can perform the standard operations of our application and some statistics wich not require high privilege; than we have the *Administrator* who can perform advanced operations on the database

- The *Standard User* can perform the following operations:
    1. Browse the list of all games
    2. Browse, search the list of games with the use of filters by name,by genres and by score and view detailed informations about them
    3. Write reviews about games composed by a comment and a score
    4. Add/Remove games from his gameList
    5. Find User by username
    6. Follow/Unfollow other users
    7. Browse the gameList of the followed users
    8. Update his user's information profile
    9. Perform some statistics

- The *Administrator* can perfom the following operations:
    1. All the precedent operations that can be performed by the standard user
    2. Create/Delete a Game
    3. Delete a Review
    4. Perform some analytics

Non Functional Requirements

- *Availability* (A): it's important in an application like this to ensure high availabilty to allow the users to continue providing services and responding to requests, even in the presence of faults or failures, so it can remains operational and accessible to users. So when the users make a write operation i update the information on the primary server and then return the control to the application in order to allow the user to continue to use the application

- *Partition* Tolerance (P): because we want to ensure the availability, we want to ensure that also in case of network partition in wich the users can still continue using the application experiencing some lost of consistencies

So we want to assure the vertices (A) and (P) of the CAP triangle and adopts the *Eventual Consistent* paradigm, because we consider this application as read-oriented application in wich we want to give privilege to fast response query by return faster the control of the applicaiton to the users

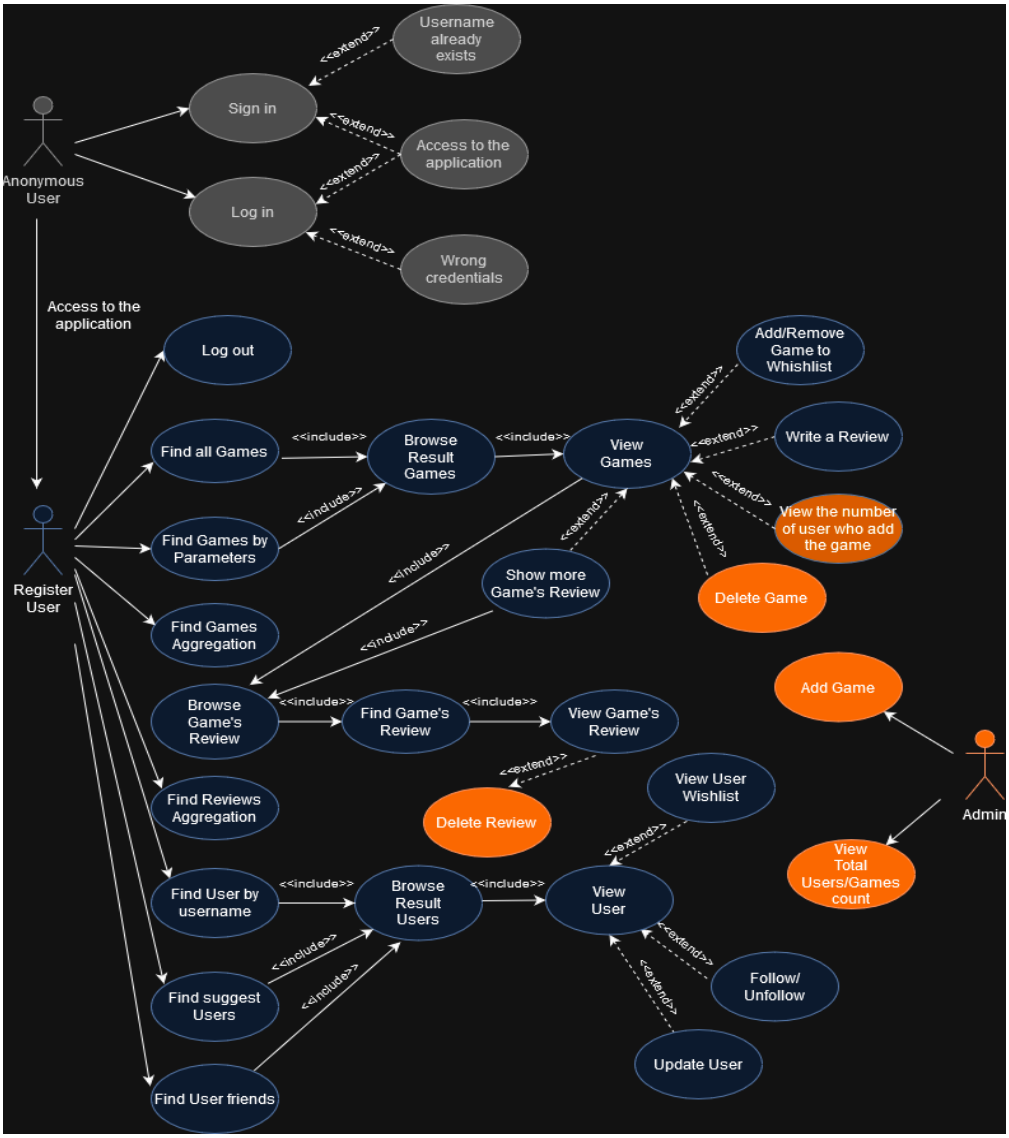
Schemas

Use-Case Diagram

In the use-case we consider the following actors:

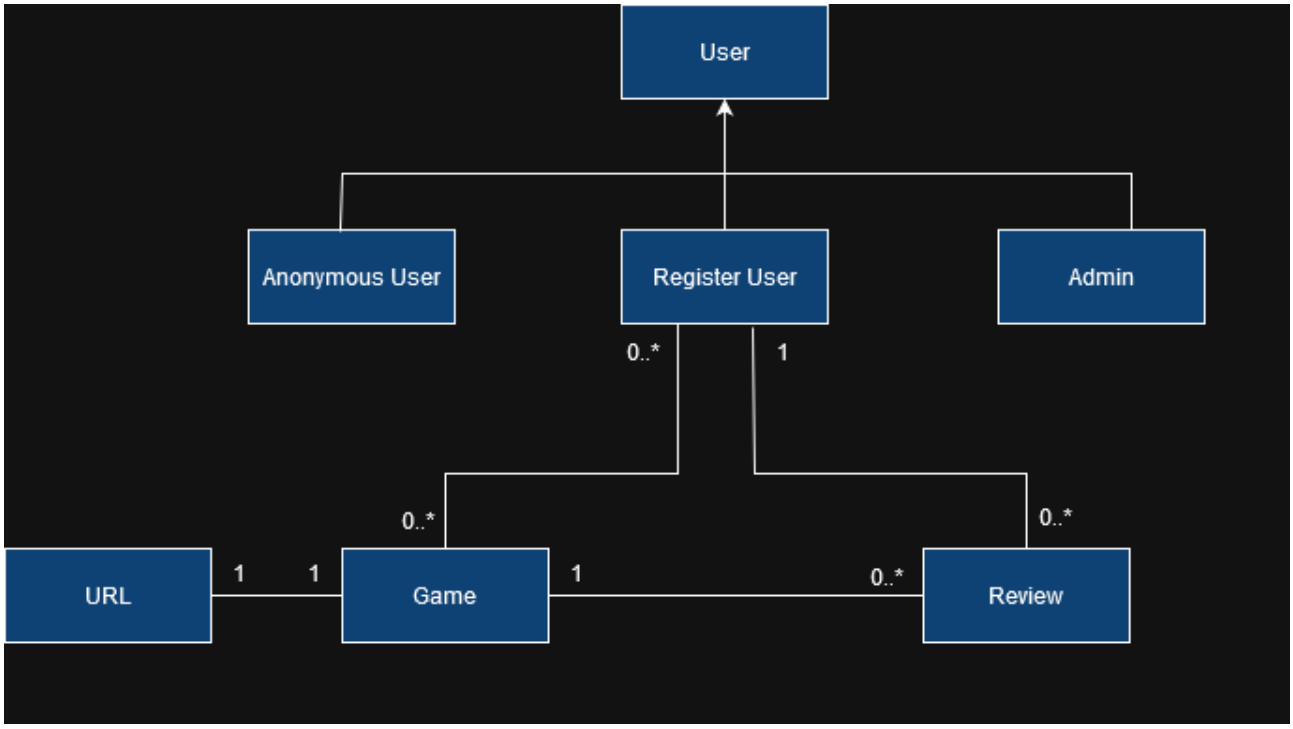
- *Anonymous User*: he is only allowed to login or signup in the application
- *Standard User*: he is allowed to perfom all the standard operations like browse and retrieve games searching by parameters, add game to their gameList, write reviews and put a like on other's users reviews, follow other users and view the gameList of their own friends and some statistics
- *Administrator*: he can make advanced functionalities like the CRUD operation on the entities and some advanced analytics

Below we see the representation of the Use-Case diagram



UML Diagram

Below we see the representation of the UML diagram and the most important fields of the entities considered and in wich database these fields are put

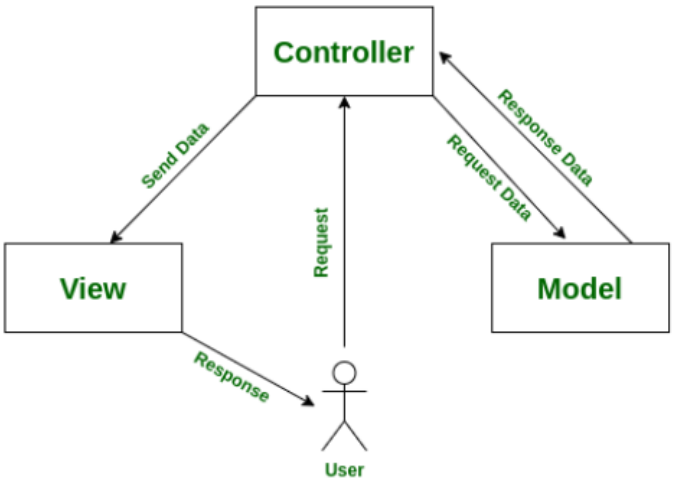


Software Architecture

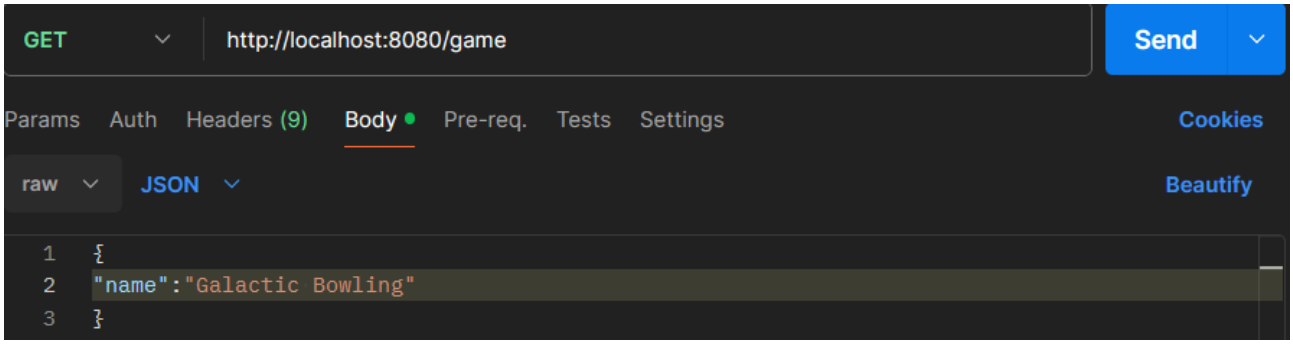
We use the Client-Server paradigm to build a Web-application by using *java 17* as programming language with *Spring Boot* framework, we use *Postman Framework* to test the query

Client Side

We use the strategy of the Model-View-Controller to implement the client side



1. The Controller is represented by the URL in wich the user can make the request to the database.



- GET represent the type of request we want to perform
- http://localhost:8080/game represents the path URL in wich the user make request
- We have the parameter that the user can specify to make some particular research

So in the controller package the annotation *@GetMapping* represents the type of request in this case a get request and inside the parenthesis *(/game)* it is give the path beginning from *http://localhost:8080* . The function wich take as input the parameter given by the user and mapped in the GameDTO class, as wee se the annotation *@RequestBody* represent the parameters to given in postman i the section *Body* in a json format

```
@GetMapping("/game")
public ResponseEntity<List<Game>> retrieveGamesByParameters(@RequestBody GameDTO gameDTO) {
    List<Game> gameList = gameService.retrieveGamesByParameters(gameDTO);
    if (!gameList.isEmpty()) {
        return ResponseEntity.ok(gameList);
    }
    System.out.println("gamelist empty");
    return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
}
```

In the controller the function take as input the DTO(Data Transfer Object) represented by this class

```
public class GameDTO {
    private String id;
    private String name;
    private String releaseDate;
    private Double price;
    private String aboutTheGame;
    private String supportedLanguages;
    private String developers;
    private String publishers;
    private String categories;
    private String genres;
    // mettere i voti
    private Integer avgScore;
}
```

2. Now the Controller interact with Service layer that have the business logic of the application

```java
public List<Game> retrieveGamesByParameters(GameDTO gameDTO) {
    try {
        if (gameDTO.getName() != null) {
            return gameRepository.findByName(gameDTO.getName());
        } else if (gameDTO.getGenres() != null && gameDTO.getAvgScore() != null) {
            // Both score and genres are provided
            return gameRepository.findByGenresAndAvgScoreGreaterThanEqual(gameDTO.getGenres(), gameDTO.getAvgScore());
        } else if (gameDTO.getGenres() != null) {
            // Only genres are provided
            return gameRepository.findByGenres(gameDTO.getGenres());
        } else if (gameDTO.getAvgScore() != null) {
            // Only score are provided
            return gameRepository.findByAvgScoreGreaterThanEqual(gameDTO.getAvgScore());
        } else {
            // No specific criteria, return empty list
            return Collections.emptyList();
        }
    } catch (Exception e) {
        System.out.println(e.getMessage());
        return null;
    }

}
```

3. Then we have the interaction between Service and Repository layer wich interact directly with the database by implements the *MongoRepository* interface

```java
@Repository
public interface GameRepository extends MongoRepository<Game,String>, GameRepositoryCustom {
    3 usages  Ludovico96
    List<Game> findByName(String name);
    1 usage  Ludovico96
    List<Game> findByGenres(String genres);
    1 usage  Ludovico96
    List<Game> findByAvgScoreGreaterThanEqual(int avgScore);
    1 usage  Ludovico96
    List<Game> findByGenresAndAvgScoreGreaterThanEqual(String genres,int avgScore);
    Ludovico96
    Page<Game> findAll(Pageable pageable);

}
```

The *MongoRepository interface* in Spring Boot provides a set of methods for interacting with a MongoDB database in a simple manner. It is part of the Spring Data MongoDB module, which aims to simplify the access to the databases in Spring applications.

Here's a brief overview of the MongoRepository interface:

1. **Basic CRUD Operations**: MongoRepository provides methods for CRUD (Create, Read, Update, Delete) operations such as *save, findById, findAll, delete*, etc. These methods allow you to perform common database operations without specify the function in the repository

2. **Query Methods**: We can define custom query methods by simply declaring them in our repository interface according to Spring Data's method naming conventions. For example, if we have a field named name in your entity class, you can define a method like *findByName(String name)* and Spring Data will automatically generate the appropriate MongoDB query.
3. **Pagination**: MongoRepository supports pagination and sorting out of the box. You can use methods like *findAll(Pageable pageable)* to retrieve data in paginated form. We use this type of function to retrieve all the games
4. **Integration with Spring Boot**: You can simply declare your repository interfaces as beans using annotations like *@Repository,@Service,@Restontroller* given the specific part of the application and Spring Boot will handle the rest

Overall, the MongoRepository interface in Spring Boot simplifies MongoDB database access by providing a consistent API and reducing the amount of code you need to write

4. Once retrieved the results it is mapped on the model class

```
@AllArgsConstructor
@NoArgsConstructor
@Getter
@Setter
@ToString
@Document(collection = "games")

@CompoundIndexes({
        @CompoundIndex(name = "genres_avgScore", def = "{'genres' : 1, 'avgScore': 1}")
})

public class Game {
    @Id
    private String id;
    @Field("name")
    private String name;
    @Indexed
    @Field("genres")
    private String genres;
    @Field("releaseDate")
    private String releaseDate;
    @Indexed
    @Field("avgScore")
    private int avgScore;
    @Field("Price")
    private double price;
    @Field("About the game")
    private String aboutTheGame;
```

In the model the we have a various number of annotations:

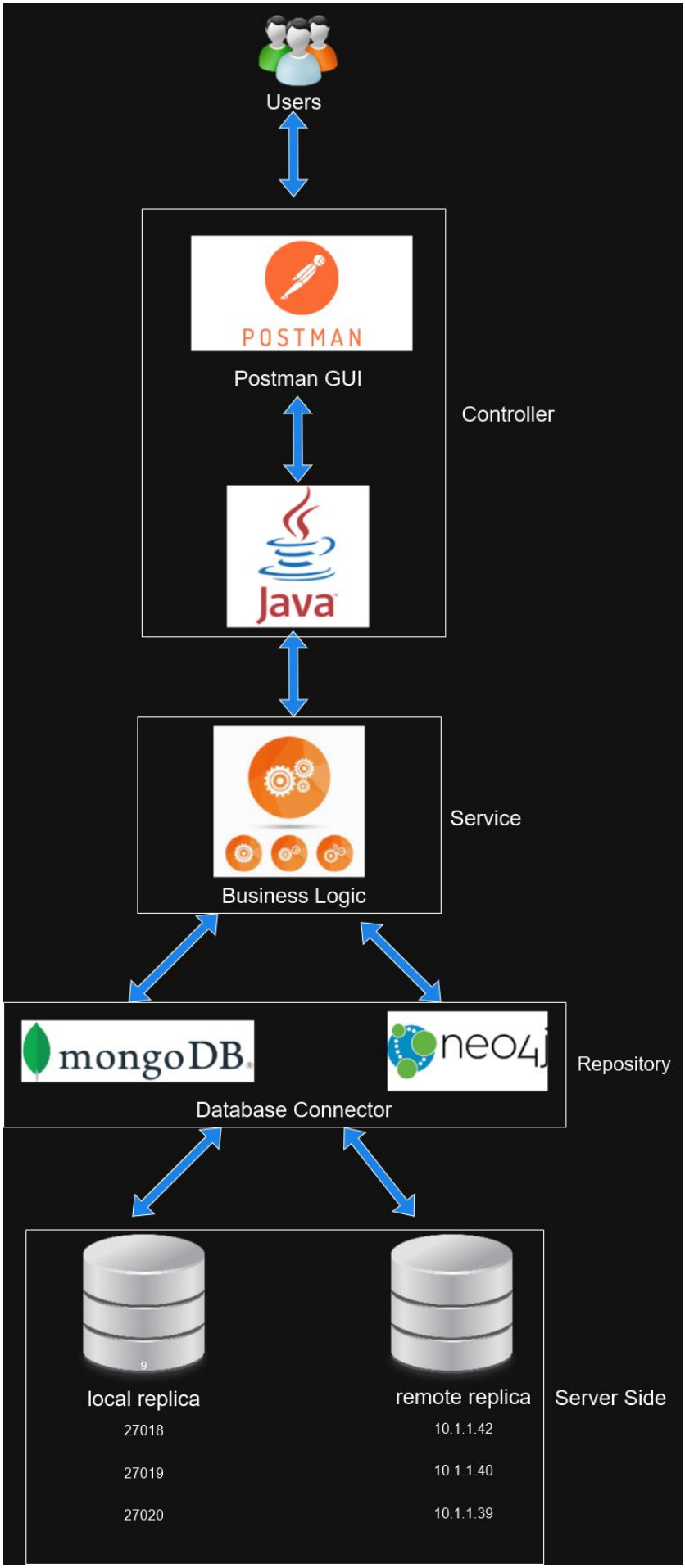
- *@ArgsConstruct* build different type of constructor
- *@Getter* and *@Setter* build all the get and set method for all the specified fields
- *@Document*(collection=games) specify the name of the collection in MongoDB
- *@CompoundIndexes* build a compound Index
- *@Id* show the _id in the database
- *@Field* show the field of the document in the database
- *@Indexed* the field for wich we build an index

5. And the view will be updated


```
[
    {
        "id": "657f3bc24fb3aa828b297c28",
        "name": "Galactic Bowling",
        "genres": "Casual,Indie,Sports",
        "releaseDate": "Oct 21, 2008",
        "avgScore": 0,
        "price": 19.99,
        "aboutTheGame": "Galactic Bowling is an exaggerated and stylized bowling game
            with an intergalactic twist. Players will engage in fast-paced single and
            multi-player competition while being submerged in a unique new universe
            filled with over-the-top humor, wild characters, unique levels, and
            addictive game play. The title is aimed at players of all ages and skill
            sets. Through accessible and intuitive controls and game-play, Galactic
            Bowling allows you to jump right into the action. A single-player campaign
            and online play allow you to work your way up the ranks of the Galactic
            Bowling League! Whether you have hours to play or only a few minutes,
            Galactic Bowling is a fast paced and entertaining experience that will leave
            you wanting more! Full Single-player story campaign including 11 Characters
            and Environments. 2 Single-player play modes including Regular and Battle
            Modes. Head to Head Online Multiplayer play Modes. Super Powers, Special
            Balls, and Whammies. Unlockable Characters, Environments, and Minigames.
```


Server Side

Talk about the replicas on virtual machine

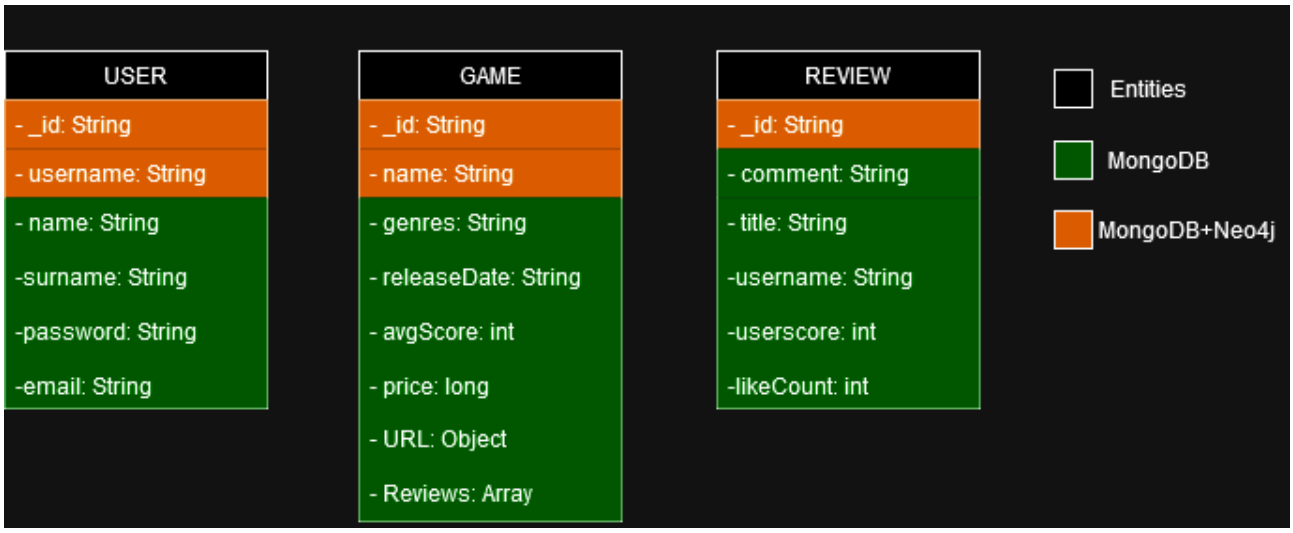

Users
POSTMAN
Postman GUI
Controller
Java
Service
Business Logic
mongoDB
neo4j
Repository
Database Connector
local replica
27018
27019
27020
remote replica
10.1.1.42
10.1.1.40
10.1.1.39
Server Side

Dataset

The Application is divided in 3 dataset:

1. A *Steam Games Dataset* wich conteining information about videogames, we take this dataset from Kaggle in a json format. Naturally we modify a little bit the structure of this database in order to use in a better way the features offered by MongoDB (83.6 k documents)
2. *A Metacritic Game Reviews Dataset* take from Kaggle (284k documents)
3. A *Users Dataset* obteined by using the usernames found in the Game Reviews Dataset, and generate the other informations about the users by the Faker library in Java (133.6 k documents)

Then we can see what entity we upload in Mongo and what we upload in Neo4j



In MongoDB, we typically include all fields of entities in documents, as it optimizes read operations by retrieving entire documents. Additionally, MongoDB's replicas ensure document redundancy, safeguarding against server failures. Conversely, in Neo4j, we store only essential fields of entities. This approach aligns with Neo4j's primary role in managing the social network aspects of applications, focusing on efficiently managing relationships between entities.

MongoDB Design and Implementation

Documents Structure

We divide the Game dataset in MongoDB in three collections:

1. *Games Collection*


```
[
  {
    _id: ObjectId("657f3bc24fb3aa828b297c28"),
    name: 'Galactic Bowling',
    genres: 'Casual,Indie,Sports',
    releaseDate: 'Oct 21, 2008',
    avgScore: 0,
    Price: 19.99,
    'About the game': 'Galactic Bowling is an exaggerated and stylized bowling game with an intergalactic twist. Player
 will engage in fast-paced single and multi-player competition while being submerged in a unique new universe filled wi
h over-the-top humor, wild characters, unique levels, and addictive game play. The title is aimed at players of all age
 and skill sets. Through accessible and intuitive controls and game-play, Galactic Bowling allows you to jump right int
 the action. A single-player campaign and online play allow you to work your way up the ranks of the Galactic Bowling L
ague! Whether you have hours to play or only a few minutes, Galactic Bowling is a fast paced and entertaining experienc
 that will leave you wanting more! Full Single-player story campaign including 11 Characters and Environments. 2 Single
player play modes including Regular and Battle Modes. Head to Head Online Multiplayer play Modes. Super Powers, Special
Balls, and Whammies. Unlockable Characters, Environments, and Minigames. Unlock all 30 Steam Achievements!',
    'Supported languages': "['English']",
    Developers: 'Perpetual FX Creative',
    Publishers: 'Perpetual FX Creative',
    Categories: 'Single-player,Multi-player,Steam Achievements,Partial Controller Support',
    URL: {
      Website: 'http://www.galacticbowling.net',
      'Header image': 'https://cdn.akamai.steamstatic.com/steam/apps/20200/header.jpg?t=1640121033',
      'Support url': '',
      'Support email': '',
      Screenshots: 'https://cdn.akamai.steamstatic.com/steam/apps/20200/0000005994.1920x1080.jpg?t=1640121033,https://c
n.akamai.steamstatic.com/steam/apps/20200/0000005993.1920x1080.jpg?t=1640121033,https://cdn.akamai.steamstatic.com/stea
/apps/20200/0000005992.1920x1080.jpg?t=1640121033,https://cdn.akamai.steamstatic.com/steam/apps/20200/0000006011.1920x1
80.jpg?t=1640121033,https://cdn.akamai.steamstatic.com/steam/apps/20200/0000005685.1920x1080.jpg?t=1640121033,https://c
n.akamai.steamstatic.com/steam/apps/20200/0000005686.1920x1080.jpg?t=1640121033,https://cdn.akamai.steamstatic.com/stea
/apps/20200/0000005995.1920x1080.jpg?t=1640121033,https://cdn.akamai.steamstatic.com/steam/apps/20200/0000005688.1920x1
80.jpg?t=1640121033,https://cdn.akamai.steamstatic.com/steam/apps/20200/0000005689.1920x1080.jpg?t=1640121033,https://c
n.akamai.steamstatic.com/steam/apps/20200/0000005690.1920x1080.jpg?t=1640121033'
    },
    Reviews: [
      {
        _id: ObjectId("6582b6cdcb6838f7c55858f5"),
        Title: 'Galactic Bowling',
        Userscore: 10,
        Comment: "            This review contains spoilers, click expand to view.        This game is do doubt the gre
test game in existance. Non, except for Nintendo haters, have been able to play this game without admiting it is at lea
t one of the best of the best, if not THE best of the best. Super Mario 64 was the first game to make 3D conversion wor
, but this was the game that perfected it. It introduced a large 3 dimensional world to explore, which at the time was
```


The game collection is given by many fields given by String,Integer and Float wich describes some informations regarding Videogames like: *name, genres, releaseDate, avgScore, Price, Developers, Publishers* and *Categories*

The *Website,Header Image,Support URL,Support Email* and *Screenshots* classes have been merged in a single *URL* field, which was chosen to improve the read performance, allowing the information to be retrieved with a single read operation

In the collection games we add the embedded field *Reviews*, in wich we upload the top 20 liked reviews, so a user when he retrieve a game can retrieve also these 20 reviews in only one read considering that in an application like this in wich the users are interested on games, they can be interested in read some reviews about those games,and because this is a kind of social in wich the Users can follow each others they can be interestered in retrieve in one scan these comments with the likeCounts to see the degree of appreciation of a user

about a game. If a User want to retrieve more reviews he should scan the Reviews Collection, but we consider that the most percentage of Users want to see only a limited number of reviews, and in genral the top scored, we decide to build this embedded field in wich we insert the top 20 reviews, a very short percentage want to scan all the Reviews Collection to retrieve all the remining Reviews associated by a title. Furthemore by upload these reviews about games we can add a sort of variety properties to our application, because we can show during a certain period of time different reviews for a given game. In this way by putting in the embedded field a constant number of colletion we can also deal with the maximum dimension of the document of 16Mb in MongoDB, so the remaining review for a given game are saved in the collection Review

2. *Reviews Collection*

```
{
  _id: ObjectId("6582b6cdcb6838f7c5585900"),
  '': 19,
  Comment: "Hands down, the best game of all time. My favorite game on any list. Right from the first moment, yo
tured into the gameplay and you don't want to stop playing. the landscape is beautiful, the monsters are something
ouldnt expect, the dungeons are challenging yet fun, and the bosses are a landmark to beat. Once you beat the game
irst time, you accually feel like you haveHands down, the best game of all time. My favorite game on any list. Rig
m the first moment, your captured into the gameplay and you don't want to stop playing. the landscape is beautiful
monsters are something you wouldnt expect, the dungeons are challenging yet fun, and the bosses are a landmark to
Once you beat the game the first time, you accually feel like you have rescued a real princess and saved a real ki
 The time playable land is massive and it will take some time to get everything, but the reward is well worth it.
down. Best. Game. Ever.",
  Title: 'Jolt Project',
  Username: 'Winky-Nichols',
  Userscore: 10,
  likeCount: 90
}
```

The fields of the document in the collection Reviews are given by String and Integer in wich we have the *comments* and the *userscore* write by the User, the name of the User, the *title* of the Game to wich the review is referring and the number of Like(*likeCount*) for that Review

3. *Users Collection*

```
{
  _id: ObjectId("6582b6ed3a6e95131310dd00"),
  username: 'marianojoey',
  name: 'Don',
  surname: 'Rolfson',
  password: 't3q6cm7vfpp',
  email: 'scot.ohara@yahoo.com'
}
```

The fileds of the colleciton Users is given only by String values wich represents the standard informations about a user like: *name, surname, username, password* and *email*

So as we see from the structure of these collections we take advantage of the structure of MongoDB by representing the one to many relationship between the entities Game and Review exploiting the *embedded form* of the DocumentDB, so under our hypothesis in wich the users are interested also in retrieval a given number of review, we can retrieve with ony one read both the games and the reviews associated to it

Mongo Queries Analysis

Now we see the queries that we perform in MongoDB and their frequency:

Read Operations		
Operation	Expected Frequency	Cost
Retrieve a Game by name	Very High	Low(1 read)
Retrieve all the Games	High	Avarage(multiple reads)
Retrieve a list of Game by parameters	High	Low(multiple reads, Indexed)
Retrieve the reviews by a title	Avarage	Avarage(multiple reads)
Retrieve list of game's genres ,ordered by avarage score and count	Low/Avarage	High(Aggregation)
Retrieve list of date Year, ordered by avarage score and count	Low/Avarage	High(Aggregation)
Retrieve the list of Users ordered by like count reviews	Low	High(Aggregation)
Retrieve the list of Game's title ordered by avarage Review's Userscore and count	Low	High(Aggregation)
Count the total number of games/users	Low	Avarage(multiple reads)
Login	Avarage	Avarage(multiple reads)

Write Operations		
Operation	Expected Frequency	Cost
Update the top 20 Reviews for a Game from scratch	Low	High(multiple reads + 1 write)
Create a Game	Low	Avarage(add a document)
Delete a Game	Very Low	Avarage(remove a docoument)
Create a Review	Low/Avarage	Avarage(add a document+1 read+some optional operations)
Delete a Review	Low	Avarage(delete a document+1 read+1 optional write)
Create/Delete/Update a User	Low	Avarage(add/remove /update a document)
Add like	Low/Avarage	Avarage(update 1 field+1 read+some optional operaiotns)
Update the top 20 embedded Reviews for a Game	Low	Avarage(1 reads+1 write)

As show in the table in a *read-oriented* application because the number of frequency of read operation is higher, so the read operation impact more on the performance of the application

Update Game's Review

The purpose of this function is to maintain the order of the embedded 'Review' field within the 'Game' document, taking into account the number of reviews for a given game or the 'likeCount' field. While this approach incurs a slight overhead in certain read operations, it ensures a kind of consistency in the embedded review list under specific circumstances. This field is updated in three different scenarios:

1. *Create a Review*:
   Initially a Game have only a field Review with one empty element, each time we call the function to create Review we should retrieve the interested Game and check if the in the embedded field there is no update reviews, so we verify if the number of reviews related to that game is higher than a given constant, if not we return the control of the application to the user with a small overhead given by some reads, otherwise we update with the function *updateGameReviewFromScratch()* (at most this function update the top 20 reviews)wich take the review and order these basing on the field likeCount and then do a write operation to update the embedded field, so in this case the overhead is given by some added reads one more read operation.

2. *Add a Like*:
   When a user add a like to a Review we retrieve the Game associated to that review, then we verify if the likeCount of this review is higher than the review in the embedded field with lesser like, if not we return the control to the applciation to the user with a small overhead given by only one read, otherwise we check if the review we are considering in inside the embedded Game's field or not, if it is inside we call the function *UpdateGameEmbeddedReview()* that consider only the reviews inside the embedded field we have already retrieved and it do only a write operation to mantein ordered these embedded reviews, otherwise if the review we are considering is outside the embedded field we call *updateGameReviewFromScratch()* that update from scratch the embedded field
3. *Delete a Review*:
   This is the function that should be runned with the lesser frequency because it is a admin function that should be invoched in really extreme cases. Also in this case the small overhead is given by the fact that when we delete a review form the collection Reviews we should check in the same review is present in the embedded field of the related game and in case, and in case eliminate the review also there

So in conclusion this logic implemented in the service layer is used to mantein a sort of consistency to mantein updated the user about the most rated reviews related to a given game by adding a small overheads given in the most percentage of the cases by only one or some read operations. If we consider that in general he frequency estimated for these two functions is substantiallly against the frequency of the standard read operations, the common sense says to us that a user can pay this little overhead for these write operations to have in some cases the consistency of this embedded reviews field and so this helps the high frequency read operations to retrieve in one read the most rated reviews with the related game

Aggregation Implementation

Custom Repository: we have to define custom repository methods beyond what MongoRepository provides, we create our own interface and extend MongoRepository. Spring Data will automatically provide implementations for methods defined in both MongoRepository and your custom interface. We create for the aggregations a structure like this in wich i have a *GameRepositoryCustom Interface* in wich we define the aggregation methods and a *GameRepositoryImpl Class* in wich i implement the methods and we see that the GameRepository interface extends both MongoRepository and the GameRepository interface

```
@Repository
public interface GameRepository extends MongoRepository<Game,String>, GameRepositoryCustom
```

Same things we do for the Review's Aggregation

In the following we have an example of code in wich we implements an aggregation

```
@Repository
public class GameRepositoryImpl implements GameRepositoryCustom {

    3 usages
    private final MongoTemplate mongoTemplate;

    no usages  Ludovico96
    @Autowired
    public GameRepositoryImpl(MongoTemplate mongoTemplate) { this.mongoTemplate = mongoTemplate; }
    1 usage  Ludovico96
    @Override
    public List<GameDTOAggregation> findAggregation() {

        GroupOperation groupByGenres = group( ...fields: "genres")
                .avg( reference: "avgScore").as( alias: "avgScore")
                .count().as( alias: "count");

        SortOperation sortByAvgScoreDesc = sort(Sort.by(Sort.Direction.DESC,  ...properties: "avgScore"));



        Aggregation aggregation = newAggregation(groupByGenres, sortByAvgScoreDesc);
        return mongoTemplate.aggregate(aggregation,  collectionName: "games", GameDTOAggregation.class).getMappedResults();


    }
```

We see that the results is mapped on a different DTO class in wich we set an annotation *@Id* wich represent in this case the new Id given by the result of the aggregation, and the other fields that will be printed in that result

```
@AllArgsConstructor
@NoArgsConstructor
@Getter
@Setter
@ToString
public class GameDTOAggregation {
    @Id
    private String genres;
    private double avgScore;
    private int count;
}
```

In the following table we see the aggregations implemented in MongoDB

Queries	MongoDB Aggregations
Retrieve the list of game's genres ,ordered by avarage score and count	`db.games.aggregate([ { $group: { _id: "$genres", avgScore: { $avg: "$avgScore" }, count: { $sum: 1 } } }, { $sort: { avgScore: -1 } } ]);`
Retrieve the list of game's date Year ,ordered by avarage score and count	`db.games.aggregate([ { $project: { year: { $year: { $toDate: "$releaseDate" } }, avgScore: 1 } }, { $group: { _id: "$year", count: { $sum: 1 }, avgScore: { $avg: "$avgScore" } } }, { $sort: { avgScore: -1 } } ]);`
Retrieve list of top 10 Users ordered by sum of the like Count Review	`db.reviews.aggregate([ { $group: { _id: "$Username", likeCount: { $sum: "$likeCount" } } }, { $sort: { likeCount: -1 } }, { $limit: 10 } ]);`
Retrieve list of Game's title ordered by avarage Reviews's Userscore and count	`db.reviews.aggregate([ { $group: { _id: "$Title", Userscore: { $avg: "$Userscore" }, count: { $sum: 1 } } }, { $sort: { Userscore: -1 } } ]);`

Index Configurations

We define two standard indexes and one compound index to improve the retrieval of the infromations and reduce the number of scan of the database, we consider reasonable the introductions of these indexes because the number of read operations is expected to be much greater than the number of write operations in wich we must modify also these indexes

```
{ v: 2, key: { genres: 1 }, name: 'genres' },
{ v: 2, key: { genres: 1, avgScore: 1 }, name: 'genres_avgScore' },
{ v: 2, key: { avgScore: 1 }, name: 'avgScore' }
```

- Index on the field Genres

```
executionStats: {
  executionSuccess: true,
  nReturned: 2269,
  executionTimeMillis: 5,
  totalKeysExamined: 2269,
  totalDocsExamined: 2269,
```

```
executionStats: {
  executionSuccess: true,
  nReturned: 2269,
  executionTimeMillis: 336,
  totalKeysExamined: 0,
  totalDocsExamined: 83560,
```

- Index on the field avgScore

```
executionStats: {
  executionSuccess: true,
  nReturned: 44,
  executionTimeMillis: 25,
  totalKeysExamined: 44,
  totalDocsExamined: 44,
```

```
executionStats: {
  executionSuccess: true,
  nReturned: 44,
  executionTimeMillis: 65,
  totalKeysExamined: 0,
  totalDocsExamined: 83560,
```

As we see in two preceding indexes we have the representations of the method *.explain("ExecutionStats")* when we do a query in the MongoBash, we can see on the left the executionStats with the use of the index and on the right without the use of the index. On the left we can see that the query examines a certain number of keys corresponding to the number of documents returned in the query with a very low time, while in the right part without a key we have to scann all the raws of the database without consulting any keys. So since we consider this applicaiton read-oriented the use of indexes here is very recommended

- Compound Index on the fields Genres and AvgScore

```
db.games.find({avgScore:{$gt:20},genres:"Action"}).hint({genres:1,avgScore:1}).explain("executionStats")
```

```
executionStats: {
  executionSuccess: true,
  nReturned: 1,
  executionTimeMillis: 0,
  totalKeysExamined: 1,
  totalDocsExamined: 1,
```

```
db.games.find({avgScore:{$gt:20},genres:"Strategy"}).hint({avgScore:1,genres:1}).explain("executionStats")
```

```
executionStats: {
  executionSuccess: true,
  nReturned: 1,
  executionTimeMillis: 1,
  totalKeysExamined: 33,
  totalDocsExamined: 1,
```

Furthermore we decide to use a compound index, above we see the two compounds indexes compare each others by using the method *.hint()* and we see that the index that scan a less number of keys is the index wich order by the quantity *genres* first and then the quantity *avgScore*.

Sharding Considerations

We can consider on split the game basing on genres basing on the fact that certain countries have more interested in some type of games, but remember that in our application we have an implemented function that aggegate by genres, so in the case of sharding when i call this function i should cross multiple servers to retrieve the result; i can do an hypothesys that this function is called a very few number of times otherwise we can say to shard by id that maybe is the simplest and more reasonable way in this type of application

Neo4j Design and Implementation

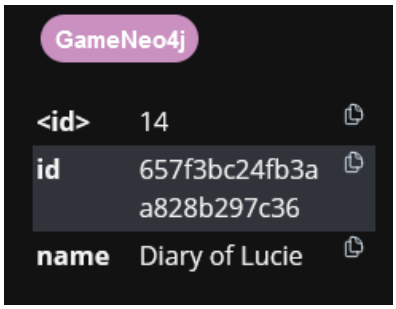
We decide to insert in Neo4j the same entites we insert in MongoDB but reducing the number of fields in order to privilege other types of operations more specific for the graph

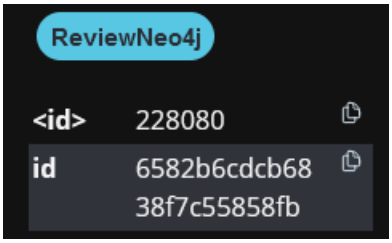
Nodes

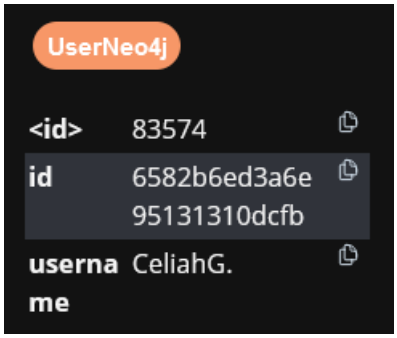
We consider three type of nodes:

1. The *UserNeo4j Node* wich represents the users of the application and in wich mantein only the field _id and the username from MongoDB
2. The *GameNeo4j Node* wich represents the games of the application in wich mantein only the field _id and the name from MongoDB
3. The *ReviewNeo4j Node* wich represents the reviews of the application in wich mantein only the field _id from MongoDB

We in the following images the nodes with their fields

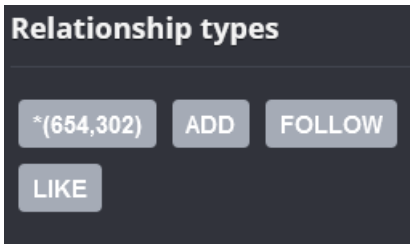




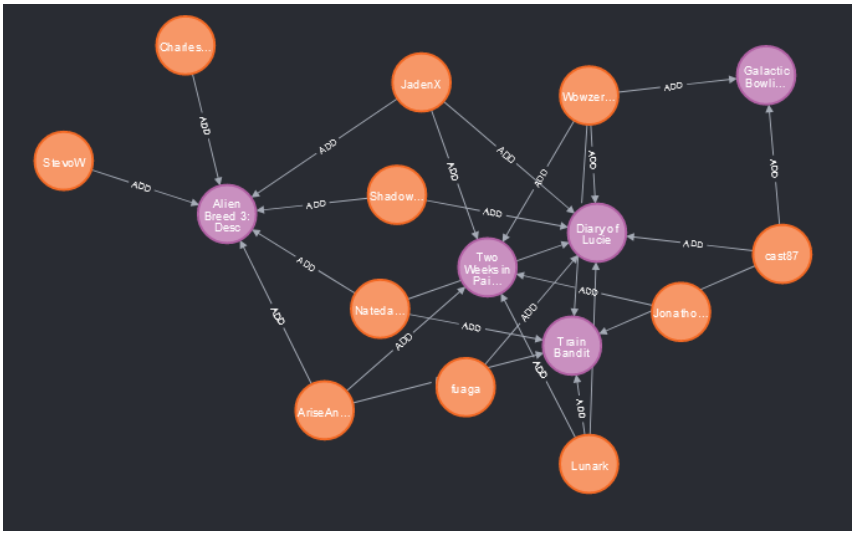


Relathionships

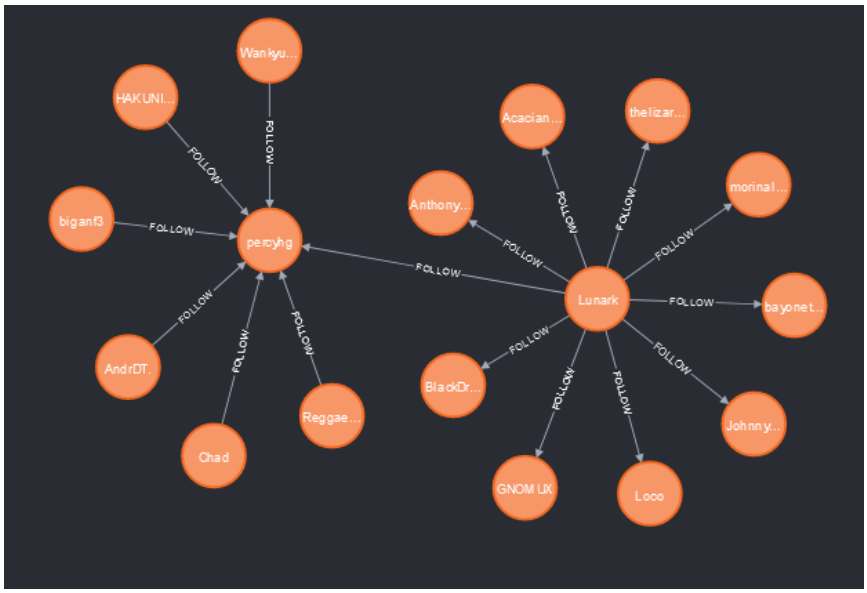
The most important reason in the choice of Neo4j is to show the relationships among the entities for fast retrieval of the informations linked between each others and handle the social network part of the application. We have more than 600k relationships divided in three type:



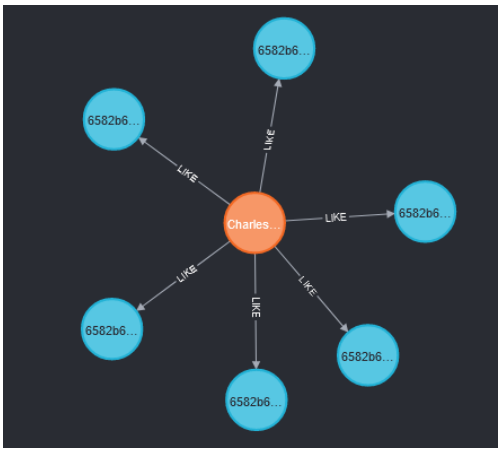
1. **User -> ADD -> Game**, wich represents the relationship between a user and his favorite games whose can be added/removed from the favourite user's game list



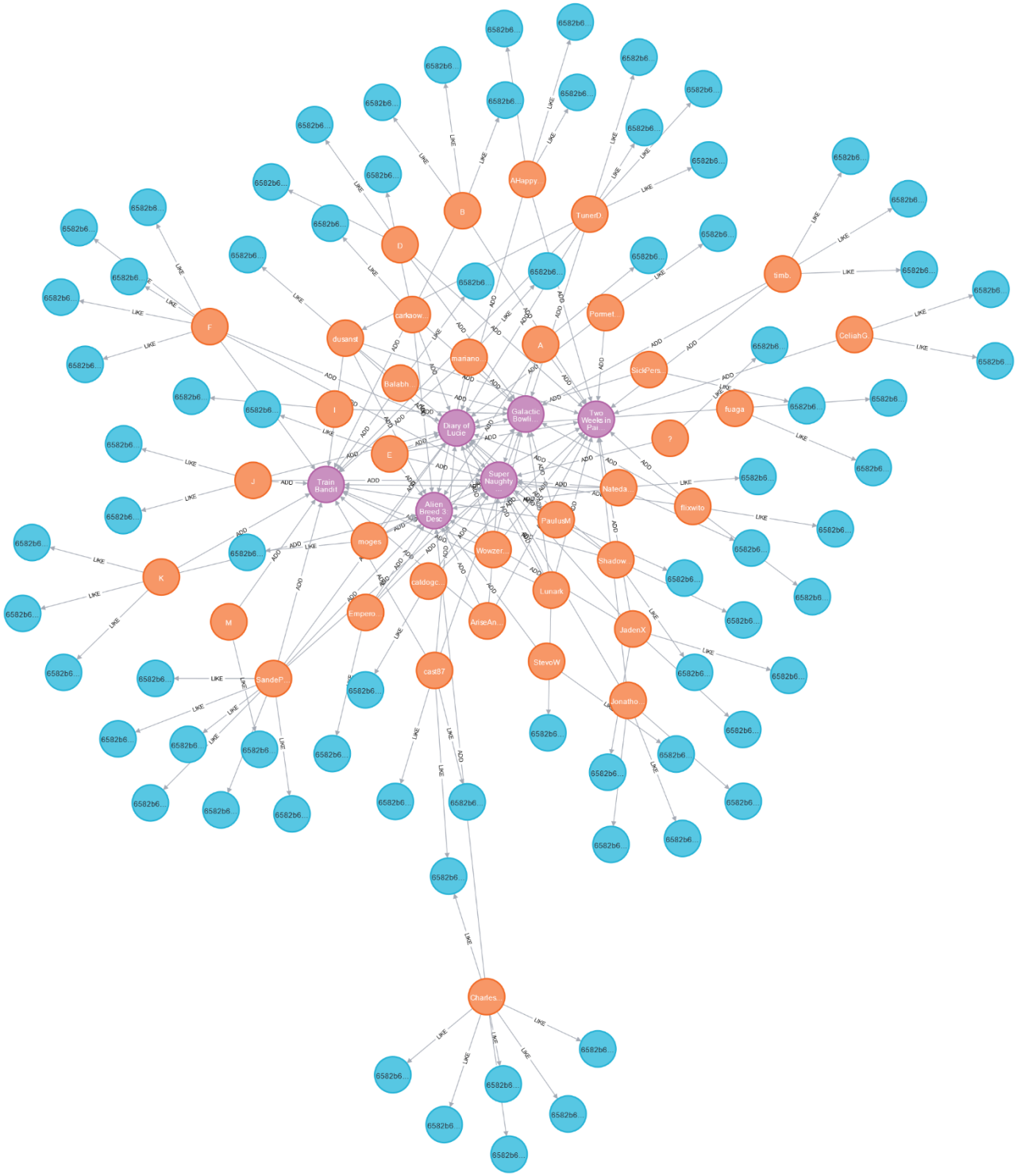
2. **User -> FOLLOW -> User**, wich represents the relaitonship between users who can became friends in the applciation by follow each others. A user can see the games added by the users he follows



3. **User ->LIKE -> Review**, wich represents the relationships betweeen a user and a review



So the entire Database result like in the following figure



Neo4j Queries Analysis

Now we see the queries that we perform in MongoDB and their frequency:

Read Operations		
Operation	Expected Frequency	Cost
Retrieve a friend's game list	High	Low(one path puntators)
Retrieve followed users	High	Low(one path puntators)
Retrieve the list of suggested friends	Low	High(multiple path puntators)
Retrieve the list of suggested games	Avarage	High(multiple path puntators)
Retrieve a user	Low	Avarage(mutliple reads)
Retrieve the ingoing ADD to game	Avarage	Avarage(one path puntators)

Write Operations		
Operation	Expected Frequency	Cost
Add/Delete game from gameList	Avarage	Low(add/delete 1 link)
Create a Game	Low	Avarage(add a node)
Delete a Game	Very Low	Avarage(remove a node)
Create a Review	Avarage	Avarage(add a node)
Delete a Review	Low	Avarage(delete a node)
Create/Delete/Update a User	Low	Avarage(add/remove /update a node)
Add LIKE to review	Avarage	Low(add 1 link)
FOLLOW/UNFOLLOW a user	Low	Low(add/remove 1 link)

We have in Neo4j more read operation against MongoDB, but also in this case the expected frequency of the read operation is predonimant

We choose to implements operation to retrieve friends and gameList's friends because of the use the structure of the graph db to retrieve quickly the result of those queries thanks to the puntators that links directly the node in the graph. Here we avoid to use index because the read operations are quickly thanks to this structore

Neo4 Queries Implementation

In the following table we see the query implemented in Neo4j:

Queries	Neo4j Queries

Retrieve the number of ingoing ADD links for a given Game	`MATCH (g:GameNeo4j)<-[:ADD]-(u:UserNeo4j)`
	`WHERE g.name = $name`
	`RETURN count(u) as numberOfLinks`

Retrieve a user by username	`MATCH (u:UserNeo4j)-[:ADD]->(g:GameNeo4j)`
	`WHERE u.username = $username`
	`RETURN g.id as id, g.name as name`

Retrieve the list of Game linked by the link ADD to a User	`MATCH (u:UserNeo4j)-[:ADD]->(g:GameNeo4j)`
	`WHERE u.username = $username`
	`RETURN g.id as id, g.name as name`

Create a link ADD from a User to a Game	`MATCH (u:UserNeo4j {username: $username}),`
	`(g:GameNeo4j {name: $name})`
	`MERGE (u)-[:ADD]->(g)`

Delete a link ADD from a User to a Game	`MATCH (u:UserNeo4j {username: $username})-[r:ADD]->`
	`(g:GameNeo4j {name: $name})`
	`DELETE r`

Retrieve the list of User linked by the link FOLLOW to a User	`MATCH (u:UserNeo4j)-[:FOLLOW]->(following:UserNeo4j)`
	`WHERE u.username = $username`
	`RETURN following`

Retrieve the list of User linked by the link FOLLOW to the Users linked by the links FOLLOW to a User	`MATCH (u:UserNeo4j {username: $username})`
	`-[:FOLLOW]->()-[:FOLLOW]->(friends)`
	`RETURN DISTINCT friends`

Create a link LIKE from a User to a Review by testing if it already exists	`MATCH (u:UserNeo4j {username: $username}),`
	`(g:ReviewNeo4j {id: $id})`
	`OPTIONAL MATCH (u)-[r:LIKE]->(g)`
	`WITH u, g, r`
	`MERGE (u)-[:LIKE]->(g)`
	`RETURN r IS NOT NULL AS relationshipExists`

We have other two queries obteined as aggregations of some of this queries

- Retrieve the Suggested Friends: in this query first we retrieve the Users linked by the links FOLLOW and the Games linked by the links ADD to the User that call the query, then we call the function to retrieve the Users linked by the links FOLLOW to the Users linked by the links FOLLOW to the User that call the function and then we see wich of them are present in the friendList of the User and we build randomly a list of a limited number of these Users. Then we retrieve the GameList(Game linked by the link ADD) of these Users and we return the User that have in common 5 or more Game with our GameList as Suggested Users. We implements this method in the service layer

```
List<UserNeo4j> getSuggestedFriends(String username);
```

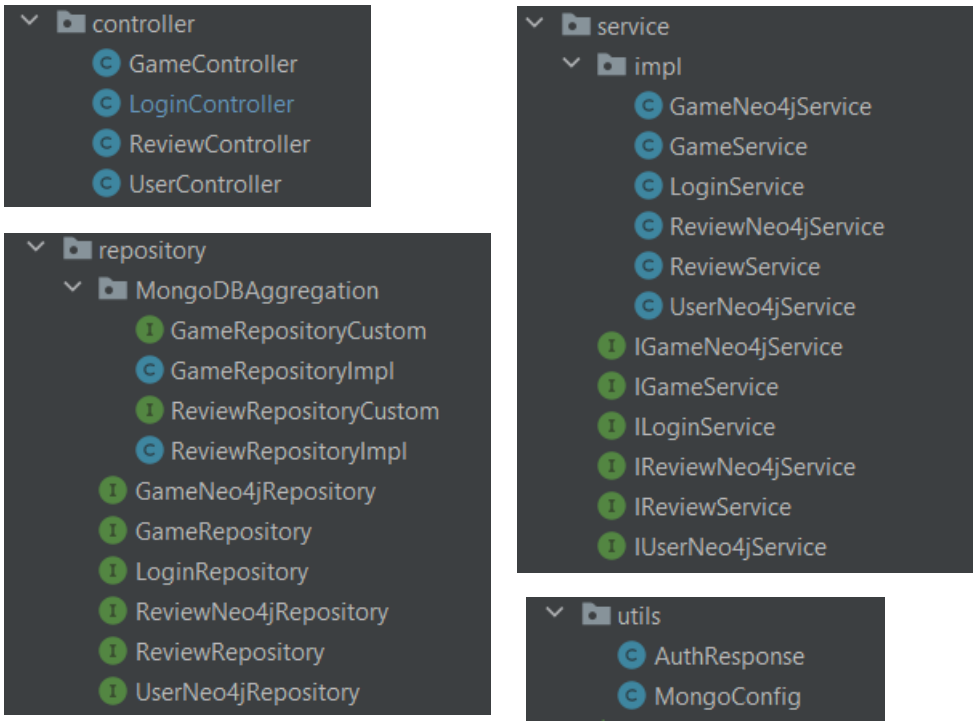
- Retrieve the Suggested Games: in this query we retrieve the Games linked by the links ADD to the User who call the query(so we retrieve his GameList) and we choose each time we call the function randomly one Game and we follow the path to the links ADD wich links the chosen Game to other Users. From this Users we retrieve their GameList and return the Games inside these GameList, because there is a greater probability that User that share common Games can have similar tastes. Otherwise if the User who call the function doesn't have any game n his gameList the function consider the Game wiht more ADD links and then retrieve the gameList of the users who ADD the most added Game

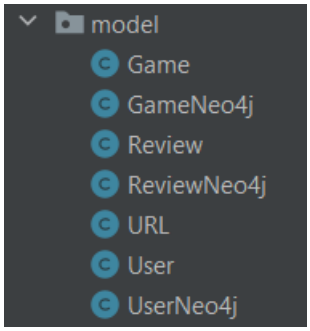
```
List<GameNeo4j> findSuggestGames(@Param("name") String name, @Param("username") String username);
```

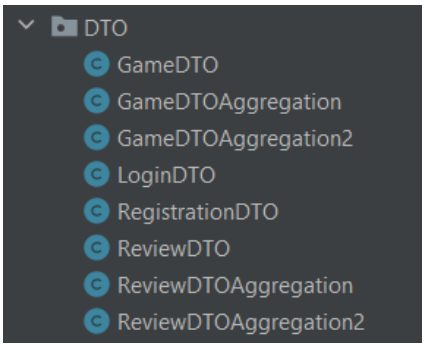
We see from these queries that we exploit the structure given by the links of the graphDB by following the path of the puntators and avoiding to scan all the graphs to retrieve the result

Structure of the Packages

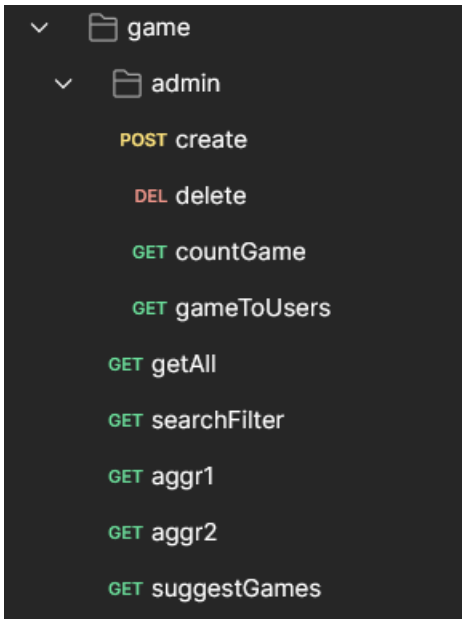
We show in the figure below the structure of the package of our code:

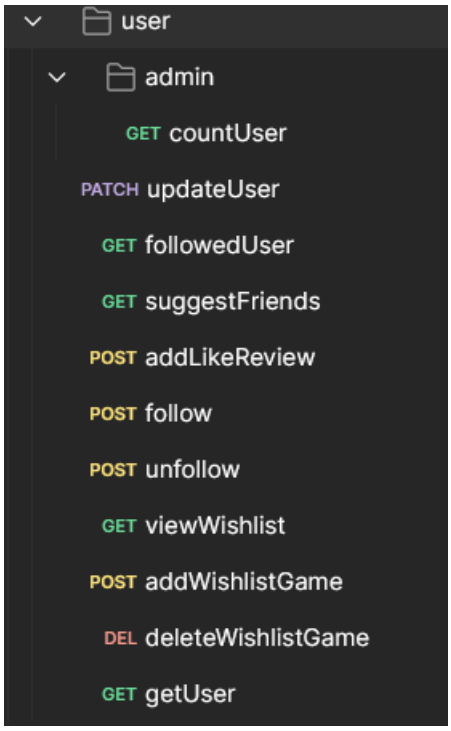


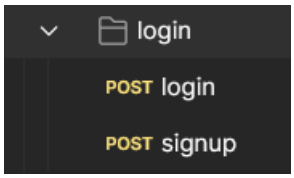


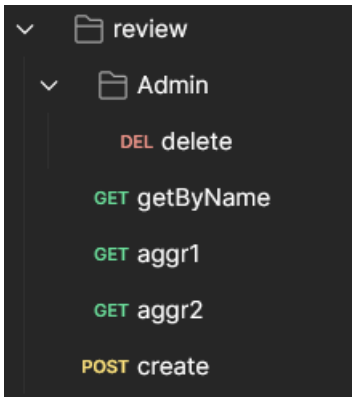


In the figure below we see the structure in wich we test the application's functionalities for different URL path









References

GameDataset: https://www.kaggle.com/datasets/fronkongames/steam-games-dataset

Review Dataset: https://www.kaggle.com/datasets/dahlia25/metacritic-video-game-comments?select=metacritic_game_user_comments.csv

GitHub link: https://github.com/tonipata/LSMSD-Project